U24AI006

ASSIGNMENT -06

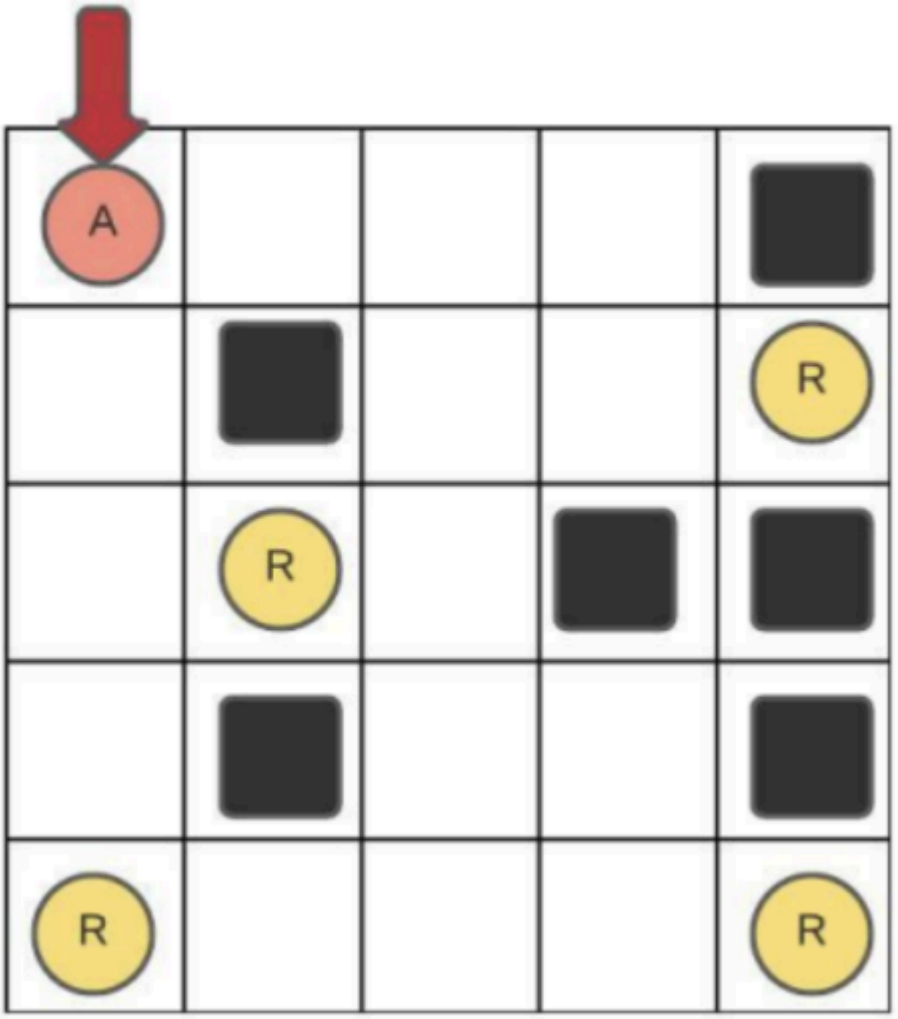
Q1 Use the city map given in Q1 of assignment 1 and implement the greedy best first search and A* algorithm to reach Boston city from Chicago and compare your results in terms of the number of cities explored before reaching your destination. Consider the heuristic values as given below,

City	h(n) to Boston (miles)
Boston	0
Providence	50
Portland	107
New York	215
Philadelphia	270
Baltimore	360
Syracuse	260
Buffalo	400
Pittsburgh	470
Cleveland	550
Columbus	640
Detroit	610
Indianapolis	780
Chicago	860

Q2 Consider a maze as a 5x5 matrix input where 0 denotes an empty tile, 1 denotes an obstruction/wall, 2 denotes the start state and 3 denotes the reward. Assume valid actions as L,R,U,D where L=move_left, R=move_right, U=move_up, D=move_down,. Use the A* algorithm on the resultant maze for your agent to reach all the rewards, and keep a record of the tiles visited on the way. Consider the rewards as goals. If all goals are completed, then one can come out of the maze. Assume heuristic cost and evaluation cost with proper justification.

U24AI006

Starting Position



U24AI006

CODE(1):

```
# list of cities (order matters)
cities = [
   "chicago", "detroit", "cleveland", "indianapolis",
   "columbus", "pittsburgh", "buffalo", "syracuse",
   "new york", "philadelphia", "baltimore",
   "providence", "boston"
]

# adjacency matrix
adj_matrix = [

# chi  det  cle  ind  col  pit  buf  syr  ny   phi  bal  pro  bos

[  0, 283, 345, 182,   0,   0,   0,   0,   0,   0,   0,   0,   0],
[283,   0, 169,   0,   0,   0, 256,   0,   0,   0,   0,   0,   0],
[345, 169,   0,   0, 144, 134,   0,   0,   0,   0,   0,   0,   0],
[182,   0,   0,   0, 176,   0,   0,   0,   0,   0,   0,   0,   0],
[  0,   0, 144, 176,   0, 185,   0,   0,   0,   0,   0,   0,   0],
[  0,   0, 134,   0, 185,   0, 215, 253,   0, 305, 247,   0,   0],
[  0, 256,   0,   0,   0, 215,   0, 150,   0,   0,   0,   0,   0],
[  0,   0,   0,   0,   0, 253, 150,   0, 254,   0,   0,   0, 312],
[  0,   0,   0,   0,   0,   0,   0, 254,   0,  97,   0, 181, 215],
[  0,   0,   0,   0,   0, 305,   0,   0,  97,   0, 101,   0,   0],
[  0,   0,   0,   0,   0, 247,   0,   0,   0, 101,   0,   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0, 181,   0,   0,   0,  50],
[  0,   0,   0,   0,   0,   0,   0, 312, 215,   0,   0,  50,   0]
]

#heuristic values
h = {
   "boston": 0, "providence": 50, "new york": 215,
   "philadelphia": 270, "baltimore": 360,
   "syracuse": 260, "buffalo": 400,
   "pittsburgh": 470, "cleveland": 550,
   "columbus": 640, "detroit": 610,
   "indianapolis": 780, "chicago": 860
}


def befs(start, goal):
   frontier = [(start, 0, [start])]
   visited = []
```

```
    explored = 0

    while frontier:

        #smallest h(n)
        min_index = 0
        for i in range(len(frontier)):
            if h[frontier[i][0]] < h[frontier[min_index][0]]:
                min_index = i

        current, cost, path = frontier.pop(min_index)

        if current in visited:
            continue

        visited.append(current)
        explored += 1

        if current == goal:
            return path, cost, explored

        current_index = cities.index(current)

        #check connected cities
        for i in range(len(cities)):
            if adj_matrix[current_index][i] != 0:
                neighbor = cities[i]

                if neighbor not in visited:
                    frontier.append(
                        (neighbor,
                         cost + adj_matrix[current_index][i],
                         path + [neighbor])
                    )

    return None


def astar(start, goal):
    frontier = [(start, 0, [start])]
    visited = []
    explored = 0

    while frontier:
```

```
        #smallest f(n) = g(n) + h(n)
        min_index = 0
        for i in range(len(frontier)):
            g1 = frontier[i][1]
            g2 = frontier[min_index][1]

            if g1 + h[frontier[i][0]] < g2 + h[frontier[min_index][0]]:
                min_index = i

        current, cost, path = frontier.pop(min_index)

        if current in visited:
            continue

        visited.append(current)
        explored += 1

        if current == goal:
            return path, cost, explored

        current_index = cities.index(current)

        #check connected cities
        for i in range(len(cities)):
            if adj_matrix[current_index][i] != 0:
                neighbor = cities[i]

                if neighbor not in visited:
                    new_cost = cost + adj_matrix[current_index][i]

                    frontier.append(
                        (neighbor,
                         new_cost,
                         path + [neighbor])
                    )

    return None


print("greedy search")
path, cost, explored = befs("chicago", "boston")
print("path:", path)
print("cost:", cost)
print("explored:", explored)
```

```
print("\na star search")
path, cost, explored = astar("chicago", "boston")
print("path:", path)
print("cost:", cost)
print("explored:", explored)
```

OUTPUT:

```
 QUESTION 1.py"
greedy search
path: ['chicago', 'cleveland', 'pittsburgh', 'syracuse', 'boston']
cost: 1044
explored: 5

a star search
path: ['chicago', 'detroit', 'buffalo', 'syracuse', 'boston']
cost: 1001
explored: 9
```

CODE (2):

```
grid = [
   [2,0,0,0,1],
   [0,1,0,0,3],
   [0,3,0,1,1],
   [0,1,0,0,1],
   [3,0,0,0,3]
]

ROWS = len(grid)
COLS = len(grid[0])

#find start and rewards
rewards = []
for r in range(ROWS):
   for c in range(COLS):
       if grid[r][c] == 2:
           start = (r,c)
       if grid[r][c] == 3:
           rewards.append((r,c))

#manhattan heuristic
def heuristic(a, b):
   return abs(a[0]-b[0]) + abs(a[1]-b[1])

#valid neighbors
def neighbors(node):
   r,c = node
   moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
    result = []
    for dr,dc in moves:
        nr,nc = r+dr, c+dc
        if 0<=nr<ROWS and 0<=nc<COLS and grid[nr][nc] != 1:
            result.append((nr,nc))
    return result

# remove lowest f(n)
def pop_lowest(frontier, f_cost):
    lowest_index = 0
    for i in range(1,len(frontier)):
        if f_cost[frontier[i]] < f_cost[frontier[lowest_index]]:
            lowest_index = i
    return frontier.pop(lowest_index)


def astar(start, goal):

    frontier = [start]
    came_from = {}
    g_cost = {start:0}
    f_cost = {start:heuristic(start,goal)}
    visited_tiles = []

    while frontier:

        current = pop_lowest(frontier, f_cost)
        visited_tiles.append(current)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1], visited_tiles

        for nb in neighbors(current):

            temp_g = g_cost[current] + 1

            if nb not in g_cost or temp_g < g_cost[nb]:
                came_from[nb] = current
                g_cost[nb] = temp_g
                f_cost[nb] = temp_g + heuristic(nb,goal)
```

```
                if nb not in frontier:
                    frontier.append(nb)

    return None, visited_tiles


#collect all rewards
current_position = start
total_path = []
all_visited = []

while rewards:

    #choose nearest reward
    nearest = rewards[0]
    min_dist = heuristic(current_position, nearest)

    for r in rewards:
        d = heuristic(current_position, r)
        if d < min_dist:
            min_dist = d
            nearest = r

    path, visited = astar(current_position, nearest)

    if path is None:
        print("No solution")
        break

    total_path += path[1:]
    all_visited += visited

    current_position = nearest
    rewards.remove(nearest)

print("Final Path:", total_path)
print("Visited Tiles:", all_visited)
print("Total Cost:", len(total_path))
```

OUTPUT:

```
Final Path: [(1, 0), (2, 0), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4), (4, 3), (3, 3), (3, 2),
(2, 2), (1, 2), (1, 3), (1, 4)]
Visited Tiles: [(0, 0), (1, 0), (0, 1), (2, 0), (2, 1), (2, 1), (2, 0), (3, 0), (4, 0), (4, 0), (4, 1), (4, 2), (4, 3
), (4, 4), (4, 4), (4, 3), (3, 3), (4, 2), (3, 2), (2, 2), (1, 2), (1, 3), (1, 4)]
Total Cost: 17
```